Exercices

◇ Partie 1 : Encapsulation

Exercice 1 :

Instructions :

1. Crée une classe `Livre` avec :
    a. `titre` (privé)
    b. `auteur` (privé)
    c. `disponible` (privé, booléen)
    d. Un constructeur pour initialiser ces valeurs.
    e. Des **getters et setters**.
    f. Une méthode `emprunter()` qui vérifie si le livre est disponible avant de l'emprunter.
    g. Une méthode `retourner()` qui remet le livre en disponibilité.
2. Crée une **méthode principale** dans `main` qui gère un livre :
    a. Demande à l'utilisateur d'entrer **le titre, l'auteur, et la disponibilité** du livre.
    b. Permets à l'utilisateur de **l'emprunter** ou **le retourner**.
    c. Affiche les informations du livre à chaque étape (après emprunt et après retour).

Exercice 2 :

Instructions :

1. Crée une classe `CompteBancaire` avec :
    a. `numeroCompte` (privé)
    b. `solde` (privé)
    c. Un constructeur et des **getters & setters**.
    d. Une méthode `deposer(double montant)` pour ajouter de l'argent.

    e. Une méthode `retirer(double montant)` qui vérifie si le solde est suffisant avant de retirer.
2. Crée une **méthode principale** dans `main` qui gère un compte bancaire :
    a. Demande à l'utilisateur de **créer un compte** avec un **numéro de compte** et un **solde initial**.
    b. Permets à l'utilisateur de **déposer de l'argent** et **de retirer de l'argent**.
    c. Affiche le **solde final** après chaque opération.

◇ Partie 2 : Héritage

Exercice 3 :

Instructions :

1. Crée une classe `Vehicule` avec :
    a. `marque` (privé)
    b. `vitesseMax` (privé)
    c. Une méthode `afficherInfo()` qui affiche la **marque** et la **vitesse max** du véhicule.
2. Crée une classe `Voiture` qui hérite de `Vehicule` et ajoute :
    a. `nombrePortes` (privé)
    b. Une méthode `afficherInfo()` qui affiche aussi le nombre de portes en plus des informations de la classe parent.
3. Crée une classe `Moto` qui hérite de `Vehicule` et ajoute :
    a. `cylindree` (privé)
    b. Une méthode `afficherInfo()` qui affiche aussi la cylindrée.
4. Crée une **méthode principale** dans `main` qui gère un véhicule :
    a. Demande à l'utilisateur de **créer une voiture ou une moto** avec les attributs appropriés (marque, vitesse, nombre de portes ou cylindrée).
    b. Affiche les informations du véhicule créé.

Exercice 4 :

Instructions :

1. Crée une classe `Employe` avec :
    a. `nom` (privé)
    b. `salaire` (privé)
    c. Une méthode `calculerSalaire()` qui retourne simplement le salaire.
2. Crée une classe `Manager` qui hérite de `Employe` et ajoute :
    a. `prime` (privée)
    b. Une méthode `calculerSalaire()` qui retourne le salaire augmenté de la prime.
3. Crée une **méthode principale** dans `main` qui gère un employé ou un manager :
    a. Demande à l'utilisateur de **créer un employé ou un manager** avec un nom, un salaire, et (si c'est un manager) une prime.
    b. Affiche le **salaire final** de l'employé ou du manager après calcul.

◇ Partie 3 : Polymorphisme

Exercice 5 :

Instructions :

1. Crée une classe **`Paiement`** avec une méthode `effectuerPaiement(double montant)` qui affiche un message générique comme `"Paiement effectué de X€"`.
2. Crée une classe **`CarteBancaire`** qui hérite de `Paiement` et redéfinit `effectuerPaiement()` pour afficher `"Paiement de X€ effectué par carte bancaire."`.
3. Crée une classe **`PayPal`** qui hérite de `Paiement` et redéfinit `effectuerPaiement()` pour afficher `"Paiement de X€ effectué via PayPal."`.
4. Crée une méthode principale dans `main` qui gère différents types de paiement :
    a. Demande à l'utilisateur **le montant du paiement** et le **type de paiement** (soit CarteBancaire ou PayPal).
    b. Instancie l'objet correspondant à **CarteBancaire** ou **PayPal** (utilise un `if` ou `switch` pour choisir le type) et appelle la méthode `effectuerPaiement()`.
    c.

Exercice 6 :

Instructions :

1. Crée une classe `Animal` avec une méthode `faireSon()` qui affiche `"Cet animal fait un bruit."`.
2. Crée deux classes **`Chien`** et **`Chat`** qui héritent de `Animal` et redéfinissent `faireSon()` :
    a. La méthode `faireSon()` de `Chien` doit afficher `"Le chien aboie : Woof!"`.
    b. La méthode `faireSon()` de `Chat` doit afficher `"Le chat miaule : Miaou!"`.
3. Crée une méthode principale dans `main` :
    a. Demande à l'utilisateur de choisir un type d'animal (soit **Chien** ou **Chat**).
    b. Instancie l'objet correspondant à **Chien** ou **Chat**.
    c. Appelle la méthode `faireSon()`.